Experiment Name:Multiple Client and a Server Communication using Thread

Theory:Multiple Client and Server Communication using Threads is a method where a single server can communicate with multiple clients simultaneously. Normally, if a server handles clients sequentially, one client's request is processed at a time, and other clients have to wait. By using threads, the server creates a separate thread for each connected client, allowing each client to communicate independently. The server uses a ServerSocket to accept incoming client connections, and each client is assigned a new thread that handles sending and receiving messages. This allows multiple clients to interact with the server at the same time without blocking each other. Such an approach is commonly used in online chat applications, multiplayer games, remote services, and banking systems. Using multi-threading ensures that the server can handle concurrency efficiently, maintain responsiveness, and provide a smooth experience for all connected clients. In short, threads enable a server to manage multiple client connections simultaneously and efficiently.

Code :
Client code:

```
package MultiClientServer;

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client {

    public static void main(String[] args) {

        try {

            Socket socket = new Socket("localhost", 5000);

            BufferedReader input = new BufferedReader(
                    new InputStreamReader(socket.getInputStream()));

            PrintWriter output = new PrintWriter(
                    socket.getOutputStream(), true);

            Scanner scanner = new Scanner(System.in);

            String message;

            while (true) {
```

```
            System.out.print("Enter message: ");
            message = scanner.nextLine();

            output.println(message);

            String response = input.readLine();
            System.out.println("Server: " + response);

            if (message.equalsIgnoreCase("bye")) {
                break;
            }
        }

        socket.close();

    } catch (Exception e) {
        System.out.println(e);
    }
  }
}
```

server code:

```
package MultiClientServer;

import java.io.*;
import java.net.*;

public class Server {

   public static void main(String[] args) {

      int port = 5000;

      try {
         ServerSocket serverSocket = new ServerSocket(port);
         System.out.println("Server started...");
         System.out.println("Waiting for clients...");

         while (true) {
            Socket socket = serverSocket.accept();
            System.out.println("Client connected");

            ClientHandler clientThread = new ClientHandler(socket);
            Thread t = new Thread(clientThread);
            t.start();
         }

      } catch (Exception e) {
         System.out.println(e);
```

```
        }
    }
}

class ClientHandler implements Runnable {

    Socket socket;

    ClientHandler(Socket socket) {
        this.socket = socket;
    }

    public void run() {

        try {
            BufferedReader input = new BufferedReader(
                    new InputStreamReader(socket.getInputStream()));

            PrintWriter output = new PrintWriter(
                    socket.getOutputStream(), true);

            String message;

            while ((message = input.readLine()) != null) {

                System.out.println("Client: " + message);

                output.println("Server received: " + message);

                if (message.equalsIgnoreCase("bye")) {
                    break;
                }
            }

            socket.close();
            System.out.println("Client disconnected");

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

output:

```
shuvo@shuvo-HP-EliteBook-840-G3:~/Documents/parral processing
penjdk-amd64/bin/java -XX:+ShowCodeDetailsInExceptionMessages
ceStorage/16f4ae2d43f86b40d099786f4a3c8cf9/redhat.java/jdt_ws
iClientServer.Client
Enter message: hello world 223311246
Server: Server received: hello world 223311246
Enter message:
```